



Exp :06 Write the python program for Vacuum Cleaner Problem.

Aim:

To write a python program for vacuum cleaner problem.

Algorithm:

Step 1: Create a data structure to represent the state of the problem, including the current position of the vacuum cleaner and the state of each square in the room (clean or dirty).

Step 2: Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.

Step 3: Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.

Step 4: At each step of the search, generate all possible successor states from the current state by applying one of the allowed actions (move up, down, left, right, or clean).

Step 5: Check if the successor state is valid (i.e., the vacuum cleaner cannot move outside the room, and it can only clean dirty squares).

Step 6: If the successor state is valid, add it to the search tree and continue the search.

Step 7: If the successor state is the goal state, return the path from the initial state to the goal state.

Step 8: If there are no more possible successor states to explore, backtrack to the previous state and try another possible action.

Program:

```
from collections import deque
```

```
def get_successors(state, rows, cols):
```

```
    vr, vc, grid = state
```

```
    successors = []
```

```

if grid[vr][vc] == 1:
    new_grid = list(list(row) for row in grid)
    new_grid[vr][vc] = 0
    successors.append(((vr, vc, tuple(tuple(row) for row in new_grid)), "CLEAN"))

```

```

directions = [(-1, 0, "UP"), (1, 0, "DOWN"), (0, -1, "LEFT"), (0, 1, "RIGHT")]

```

```

for dr, dc, action in directions:
    nr, nc = vr + dr, vc + dc
    if 0 <= nr < rows and 0 <= nc < cols:
        successors.append(((nr, nc, grid), action))

```

```

return successors

```

```

def is_goal(state):

```

```

    grid = state[2]
    for row in grid:
        if 1 in row:
            return False
    return True

```

```

def solve_vacuum_cleaner(start_r, start_c, initial_grid):

```

```

    rows = len(initial_grid)
    cols = len(initial_grid[0])

```

```

    start_state = (start_r, start_c, tuple(tuple(row) for row in initial_grid))
    queue = deque([(start_state, [])])
    visited = set([start_state])

```

```

while queue:
    current_state, path = queue.popleft()

```

```

    if is_goal(current_state):
        return path

    for next_state, action in get_successors(current_state, rows, cols):
        if next_state not in visited:
            visited.add(next_state)
            queue.append((next_state, path + [(action, next_state)]))

    return None

if __name__ == '__main__':
    room_grid = [[1, 0], [1, 1]]
    vacuum_start_row = 0
    vacuum_start_col = 0

    print("Initial Room State:")
    for row in room_grid:
        print(row)
    print("Vacuum starts at:", (vacuum_start_row, vacuum_start_col))
    print("-" * 30)

    solution_path = solve_vacuum_cleaner(vacuum_start_row, vacuum_start_col, room_grid)

    if solution_path:
        for step_idx, (action_taken, resulting_state) in enumerate(solution_path):
            r, c, current_grid = resulting_state
            print("Step", step_idx + 1, "- Action:", action_taken)
            for row in current_grid:
                print(list(row))
            print("Vacuum now at:", (r, c))

```

```

        print("-" * 30)
    else:
        print("No solution found to clean the room.")

```

OUTPUT:

```

--
Initial Room State:
[1, 0]
[1, 1]
Vacuum starts at: (0, 0)
-----
Step 1 - Action: CLEAN
[0, 0]
[1, 1]
Vacuum now at: (0, 0)
-----
Step 2 - Action: DOWN
[0, 0]
[1, 1]
Vacuum now at: (1, 0)
-----
Step 3 - Action: CLEAN
[0, 0]
[0, 1]
Vacuum now at: (1, 0)
-----
Step 4 - Action: RIGHT
[0, 0]
[0, 1]
Vacuum now at: (1, 1)
-----
Step 5 - Action: CLEAN
[0, 0]
[0, 0]
Vacuum now at: (1, 1)
-----

```

Result: Thus, the program was executed and found that the output was found to be correct.